

Real-Time Examples of Liskov Substitution Principle in C#

In this article, I will discuss **Multiple Real-Time Examples of the Liskov Substitution Principle (LSP) in C#**. Please read our previous article discussing the basic concepts of the [Liskov Substitution Principle \(LSP\) in C#](#). At the end of this article, you will understand the following Real-time Examples using the Liskov Substitution Principle (LSP) in C#.

1. [Creating Different Shapes](#)
2. [Bank Accounts](#)
3. [Vehicles and Their Engines](#)
4. [Animals and Their Sounds](#)
5. [Document Processing](#)
6. [Managing Employees in a Company](#)

Note: First, I will show the example without following the Liskov Substitution Principle (LSP). Then, I will discuss the problem of not following the Liskov Substitution Principle and write the same example following the LSP.

How to Use the Liskov Substitution Principle in C#?

Using the Liskov Substitution Principle (LSP) in C# involves designing your classes and inheritance hierarchies to ensure derived classes can be substituted for their base classes without causing unexpected behavior. Here's a step-by-step guide on how to use the Liskov Substitution Principle effectively in C#:

- **Identify the Base Class or Interface:** Identify the base class or interface that defines the common contract or behavior that derived classes should adhere to.
- **Follow the Contract:** Ensure derived classes implement the same contract or behavior defined by the base class or interface. This includes implementing methods, properties, and behavior as specified.
- **Override Methods Carefully:** When overriding methods in derived classes, make sure that the overridden methods adhere to the expected behavior defined by the base class. Avoid changing the semantics of the method.
- **No Weakening Preconditions:** Ensure that preconditions (parameters, constraints, etc.) required by the base class methods are not weakened in derived class methods. Derived class methods should meet or strengthen the preconditions.
- **No Strengthening Postconditions:** Derived class methods should not strengthen the base class methods' postconditions (return values, effects, etc.). The behavior should be consistent or more relaxed, but not more strict.
- **Avoid Empty Overrides:** Avoid overriding methods in derived classes with empty or no-op implementations. This violates the LSP because the derived class is not substitutable for the base class.
- **Use Polymorphism:** Utilize polymorphism to treat objects of derived classes as objects of the base class. This allows you to interchangeably use different implementations without affecting correctness.
- **Test Substitutability:** Test the substitutability of derived classes by using them in contexts where base class instances are expected. Ensure that the behavior remains consistent and expected.
- **Document Contracts:** Document base classes and interfaces' contracts (interfaces, abstract methods, behavior). This helps developers understand the expected behavior when implementing derived classes.
- **Refactor for Consistency:** If you encounter violations of the LSP, refactor your code to ensure that the derived classes adhere to the contract and behavior defined by the base class.

Real-Time Example of Liskov Substitution Principle in C#: Creating Different Shapes

Let's see one real-time example involving Rectangle and Square classes, one of the best examples to understand the Liskov Substitution Principle (LSP).

Violating LSP

Let us first see how we can implement the above example without following the Liskov Substitution Principle (LSP) in C#:

```
namespace LSPDemo
{
    public class Rectangle
    {
        public virtual double Width { get; set; }
        public virtual double Height { get; set; }
        public double GetArea()
        {
            return Width * Height;
        }
    }
    public class Square : Rectangle
    {
        public override double Width
        {
            get { return base.Width; }
            set { base.Width = base.Height = value; }
        }
        public override double Height
        {
            get { return base.Height; }
            set { base.Width = base.Height = value; }
        }
    }
}
```

As you can see in the above code, there is a class named Square that is a subclass of the Rectangle class and tries to enforce the rule to ensure that its width and height are equal. However, this approach violates the Liskov Substitution Principle (LSP) because if a method is intended to work with a Rectangle object, it may not function correctly when a Square instance is used as input. For a better understanding, please have a look at the following example.

```
using System;
namespace LSPDemo
{
    public class Rectangle
    {
        public virtual double Width { get; set; }
        public virtual double Height { get; set; }
        public double GetArea()
```

```

    {
        return Width * Height;
    }
    public void ChangeDimensions(Rectangle rect, double width, double height)
    {
        rect.Width = width;
        rect.Height = height;
        Console.WriteLine($"Area: {rect.GetArea()}");
    }
}
public class Square : Rectangle
{
    public override double Width
    {
        get { return base.Width; }
        set { base.Width = base.Height = value; }
    }
    public override double Height
    {
        get { return base.Height; }
        set { base.Width = base.Height = value; }
    }
}

public class Program
{
    public static void Main()
    {
        var rect = new Rectangle { Width = 2, Height = 3 };
        rect.ChangeDimensions(rect, 4, 5); // This works fine
        var square = new Square { Width = 2 };
        rect.ChangeDimensions(square, 4, 5); // This behaves unexpectedly
        Console.ReadKey();
    }
}
}

```

In the above example, when we try to set different values for the width and height of a square, the behavior is not what we expect because the area will not be 4 * 5.

With LSP

Instead of trying to make Square a subclass of Rectangle, the classes can be refactored to follow the Liskov Substitution Principle in C#. Let us see how we can implement the above example following the Liskov Substitution Principle in C#:

```

using System;
namespace LSPDemo
{
    public abstract class Shape

```

```

{
    public abstract double GetArea();
}
public class Rectangle : Shape
{
    public double Width { get; set; }
    public double Height { get; set; }
    public override double GetArea()
    {
        return Width * Height;
    }
    public void ChangeDimensions(Rectangle rect, double width, double height)
    {
        rect.Width = width;
        rect.Height = height;
        Console.WriteLine($"Area: {rect.GetArea()}");
    }
}
public class Square : Shape
{
    public double Side { get; set; }
    public override double GetArea()
    {
        return Side * Side;
    }
}
//Testing the Liskov Substitution Principle
public class Program
{
    public static void Main()
    {
        var rect = new Rectangle { Width = 2, Height = 3 };
        rect.GetArea(); // This works fine
        rect.ChangeDimensions(rect, 4, 5); // This works fine
        var square = new Square { Side = 2 };
        square.GetArea(); // This also works fine
        // This will not work
        //square.ChangeDimensions(square, 4, 5);
        Console.ReadKey();
    }
}
}

```

With this design, both the Rectangle and Square subclasses inherit from the common base Shape class. This guarantees that there are no unexpected behaviors or differences when calculating their respective areas and the Liskov Substitution Principle is followed. As a result, any method created for the Shape class can anticipate consistent behavior from both Rectangle and Square instances with no unexpected side effects.

Real-Time Example of Liskov Substitution Principle in C#: Bank Accounts

Let's see another real-time example of Bank Accounts to understand the Liskov Substitution Principle (LSP). Suppose you have two types of bank accounts: a `RegularAccount` and a `FixedTermDepositAccount`. The `FixedTermDepositAccount` doesn't allow withdrawals until the end of the term.

Violating LSP

Let us first see how we can implement the above example without following the Liskov Substitution Principle (LSP) in C#:

```
using System;
namespace LSPDemo
{
    public class BankAccount
    {
        protected double balance;
        public virtual void Deposit(double amount)
        {
            balance += amount;
        }
        public virtual void Withdraw(double amount)
        {
            if (balance >= amount)
            {
                balance -= amount;
            }
            else
            {
                throw new InvalidOperationException("Insufficient funds");
            }
        }
        public double GetBalance()
        {
            return balance;
        }
    }
    public class FixedTermDepositAccount : BankAccount
    {
        public override void Withdraw(double amount)
        {
            throw new InvalidOperationException("Cannot withdraw from a fixed deposit account until term ends");
        }
    }
}
```

If a `FixedTermDepositAccount` object is substituted for a `BankAccount` object in a context that expects withdrawals to be possible, the program will break, violating the Liskov Substitution Principle (LSP).

Following LSP:

Let's refactor the classes to follow the Liskov Substitution Principle (LSP). Let us see how we can implement the above example following the Liskov Substitution Principle in C#:

```
using System;
namespace LSPDemo
{
    public abstract class BankAccount
    {
        protected double balance;
        public virtual void Deposit(double amount)
        {
            balance += amount;
            Console.WriteLine($"Deposit: {amount}, Total Amount: {balance}");
        }
        public abstract void Withdraw(double amount);
        public double GetBalance()
        {
            return balance;
        }
    }
    public class RegularAccount : BankAccount
    {
        public override void Withdraw(double amount)
        {
            if (balance >= amount)
            {
                balance -= amount;
                Console.WriteLine($"Withdraw: {amount}, Balance: {balance}");
            }
            else
            {
                Console.WriteLine($"Trying to Withdraw: {amount}, Insufficient
Available Funds: {balance}");
            }
        }
    }
    public class FixedTermDepositAccount : BankAccount
    {
        private bool termEnded = false; // simplification for the example
        public override void Withdraw(double amount)
        {
            if (!termEnded)
            {
                Console.WriteLine("Cannot withdraw from a fixed term deposit :
until term ends");
            }
            else if (balance >= amount)
            {
                balance -= amount;
                Console.WriteLine($"Withdraw: {amount}, Balance: {balance}");
            }
        }
    }
}
```

```

        else
        {
            Console.WriteLine($"Trying to Withdraw: {amount}, Insufficient
Available Funds: {balance}");
        }
    }
}

//Testing the Liskov Substitution Principle
public class Program
{
    public static void Main()
    {
        Console.WriteLine("RegularAccount:");
        var RegularBankAccount = new RegularAccount();
        RegularBankAccount.Deposit(1000);
        RegularBankAccount.Deposit(500);
        RegularBankAccount.Withdraw(900);
        RegularBankAccount.Withdraw(800);
        Console.WriteLine("\nFixedTermDepositAccount:");
        var FixedTermDepositBankAccount = new FixedTermDepositAccount();
        FixedTermDepositBankAccount.Deposit(1000);
        FixedTermDepositBankAccount.Withdraw(500);

        Console.ReadKey();
    }
}

```

When designing the BankAccount class, we make the Withdraw method abstract to enable derived types might have their own rules for withdrawal. This approach helps the client code understand that subclasses may have varying behaviors, and Liskov Substitution Principle (LSP) is not violated. When you run the above code, you will get the following output.

```

RegularAccount:
Deposit: 1000, Total Amount: 1000
Deposit: 500, Total Amount: 1500
Withdraw: 900, Balance: 600
Trying to Withdraw: 800, Insufficient Funds, Available Funds: 600

FixedTermDepositAccount:
Deposit: 1000, Total Amount: 1000
Cannot withdraw from a fixed term deposit account until term ends

```

Real-Time Example of Liskov Substitution Principle in C#: Vehicles and Their Engines

Let's see another real-time example, Vehicles and Their Engines, to understand the Liskov Substitution Principle (LSP). Vehicles can have different types of engines: gasoline, electric, or hybrid. Each engine type has a method to start it. However, while gasoline engines use ignition and fuel, electric engines need a battery check.

Violating LSP

Let us first see how we can implement the above example without following the Liskov Substitution Principle (LSP) in C#:

```
using System;
namespace LSPDemo
{
    public class Vehicle
    {
        public virtual void StartEngine()
        {
            Console.WriteLine("Starting engine using ignition and fuel.");
        }
    }
    public class ElectricVehicle : Vehicle
    {
        public override void StartEngine()
        {
            Console.WriteLine("Checking battery and starting electric motor.");
        }
    }
}
```

When working with the base Vehicle class, it's important to note that the method StartEngine assumes an ignition mechanism. However, ElectricVehicle overrides this method to provide its own mechanism. If the client assumes that all vehicles start using an ignition mechanism, replacing the Vehicle with an ElectricVehicle would be incorrect and violate the Liskov Substitution Principle (LSP).

With LSP

Let us see how we can implement the above example following the Liskov Substitution Principle in C#:

```
using System;
namespace LSPDemo
{
    public abstract class Vehicle
    {
        public abstract void StartEngine();
    }
    public class GasolineVehicle : Vehicle
    {
        public override void StartEngine()
        {
            Console.WriteLine("Starting engine using ignition and fuel.");
        }
    }
}
```

```

    }
}
public class ElectricVehicle : Vehicle
{
    public override void StartEngine()
    {
        Console.WriteLine("Checking battery and starting electric motor.");
    }
}

//Testing the Liskov Substitution Principle
public class Program
{
    public static void Main()
    {
        Vehicle vehicle = new GasolineVehicle();
        vehicle.StartEngine();
        vehicle = new ElectricVehicle();
        vehicle.StartEngine();
        Console.ReadKey();
    }
}
}

```

Each type of vehicle now has its own StartEngine implementation, adhering to the Liskov Substitution Principle. When a client wants to start a vehicle engine, they don't need to know the specific engine type. The StartEngine method will work correctly regardless of whether the engine is powered by gasoline or electricity. This means that one type of vehicle can be swapped with another without affecting the correctness of the program, following the Liskov Substitution Principle.

Real-Time Example of Liskov Substitution Principle in C#: Animals and Their Sounds

Let's see another real-time example, Animals and Their Sounds, to understand the Liskov Substitution Principle (LSP). Animals can make different sounds. A dog barks, a cat meows, and so on.

Violating LSP

Let us first see how we can implement the above example without following the Liskov Substitution Principle (LSP) in C#:

```

using System;
namespace LSPDemo
{
    public class Animal
    {
        public virtual void MakeSound()
        {
            Console.WriteLine("Some generic animal sound.");
        }
    }
}

```

```

    }
}
public class Dog : Animal
{
    public override void MakeSound()
    {
        Console.WriteLine("Bark!");
    }
}
public class Cat : Animal
{
    public override void MakeSound()
    {
        // Cats have a special way to communicate!
        throw new NotImplementedException("Cats use telepathy. Just kidding, I haven't implemented this.");
    }
}
}
}

```

In this design, providing a Cat object to a function that expects an Animal object and tries to make it produce a sound would cause an exception, violating LSP.

With LSP

Let us see how we can implement the above example following the Liskov Substitution Principle in C#:

```

using System;
namespace LSPDemo
{
    public abstract class Animal
    {
        public abstract void MakeSound();
    }
    public class Dog : Animal
    {
        public override void MakeSound()
        {
            Console.WriteLine("Bark!");
        }
    }
    public class Cat : Animal
    {
        public override void MakeSound()
        {
            Console.WriteLine("Meow!");
        }
    }
}

//Testing the Liskov Substitution Principle

```

```

public class Program
{
    public static void Main()
    {
        Animal animal = new Dog();
        animal.MakeSound();
        animal = new Cat();
        animal.MakeSound();
        Console.ReadKey();
    }
}

```

In this example, the Animal base class defines the expectation that any derived animal type can make a sound, with the Dog and Cat classes providing their respective implementations.

Using the Liskov Substitution Principle, any derived type of Animal, such as a Dog or Cat, can be used with a client function that requires an Animal object to produce a sound, ensuring reliable functionality. For a better understanding, please have a look at the below example.

```

using System;
namespace LSPDemo
{
    public abstract class Animal
    {
        public abstract void MakeSound();
        public void MakeAnimalSound(Animal animal)
        {
            animal.MakeSound();
        }
    }
    public class Dog : Animal
    {
        public override void MakeSound()
        {
            Console.WriteLine("Bark!");
        }
    }
    public class Cat : Animal
    {
        public override void MakeSound()
        {
            Console.WriteLine("Meow!");
        }
    }

    //Testing the Liskov Substitution Principle
    public class Program
    {
        public static void Main()
        {

```

```

        Animal animal = new Dog();
        animal.MakeSound();
        animal = new Cat();
        animal.MakeSound();
        animal.MakeAnimalSound(new Dog()); // Outputs: Bark!
        animal.MakeAnimalSound(new Cat()); // Outputs: Meow!
        Console.ReadKey();
    }
}
}

```

By following LSP, we ensure that derived classes enhance or extend functionalities but don't replace or break the expected behavior of the base class.

Real-Time Example of Liskov Substitution Principle in C#: Document Processing

Let's see another real-time example of Document Processing to understand the Liskov Substitution Principle (LSP). Imagine you are designing a system that processes documents. Some documents are read-only, and others are editable.

Violation of Liskov Substitution Principle:

If you use a base Document class with both Read and Write operations, then a ReadOnlyDocument subclass might override the Write operation to prevent writing, which would violate LSP.

```

using System;
namespace LSPDemo
{
    public class Document
    {
        public virtual string Content { get; set; }
        public void Read()
        {
            Console.WriteLine(Content);
        }
        public virtual void Write(string content)
        {
            Content = content;
        }
    }
    public class ReadOnlyDocument : Document
    {
        public override void Write(string content)
        {
            throw new InvalidOperationException("Cannot write to a read-only document");
        }
    }

    //Testing the Liskov Substitution Principle
    public class Program

```

```

{
    public static void Main()
    {
        Document doc = new ReadOnlyDocument();
        doc.Write("New Content"); // This will throw an exception!
        Console.ReadKey();
    }
}

```

With Liskov Substitution Principle:

A better approach is to have separate interfaces for reading and writing and to implement those interfaces as appropriate.

```

using System;
namespace LSPDemo
{
    public interface IReadable
    {
        string Content { get; }
        void Read();
    }
    public interface IWritable
    {
        string Content { get; set; }
        void Write(string content);
    }
    public class EditableDocument : IReadable, IWritable
    {
        public string Content { get; set; }
        public void Read()
        {
            Console.WriteLine(Content);
        }
        public void Write(string content)
        {
            Content = content;
        }
    }
    public class ReadOnlyDocument : IReadable
    {
        public ReadOnlyDocument(string content)
        {
            Content = content;
        }
        public string Content { get; }
        public void Read()
        {
            Console.WriteLine(Content);
        }
    }
}

```

```

    }
}

//Testing the Liskov Substitution Principle
public class Program
{
    public static void Main()
    {
        EditableDocument editableDoc = new EditableDocument();
        editableDoc.Write("Editable Content");
        editableDoc.Read();
        ReadOnlyDocument readOnlyDoc = new ReadOnlyDocument("Read-Only Cor
        readOnlyDoc.Read();
        //readOnlyDoc.Write("Editable Content");
        Console.ReadKey();
    }
}
}

```

By not altering the expected behavior of inherited methods, this design adheres to the Liskov Substitution Principle (LSP). The `ReadOnlyDocument` class implements its functionalities through interfaces instead of having a `Write` method. This guarantees that clients who anticipate a writable document will not unintentionally receive a read-only one.

Real-Time Example of Liskov Substitution Principle in C#: Managing Employees in a Company

Let's see another real-time example of Managing Employees in a Company to understand the Liskov Substitution Principle (LSP). A company has different types of employees: regular Employees and Interns. Every employee has a salary, but interns do not receive bonuses.

Violation of Liskov Substitution Principle:

If we design a base `Employee` class that includes a `CalculateBonus` method, then the `Intern` subclass might override or handle this method differently, leading to a violation of LSP.

```

using System;
namespace LSPDemo
{
    public class Employee
    {
        public virtual double Salary { get; set; }
        public virtual double CalculateBonus()
        {
            return Salary * 0.1;
        }
    }
    public class Intern : Employee
    {

```

```

        public override double CalculateBonus()
        {
            // Interns do not get bonuses.
            return 0;
        }
    }
}
//Testing the Liskov Substitution Principle
public class Program
{
    public static void DisplayBonus(Employee employee)
    {
        Console.WriteLine($"Bonus: {employee.CalculateBonus()}");
    }
    public static void Main()
    {
        Employee emp = new Intern { Salary = 5000 };
        DisplayBonus(emp); // This might lead to confusion or unexpected behavior
        Console.ReadKey();
    }
}
}

```

In the above example, if the Intern class is substituted where Employee is expected, it alters the expected behavior of bonus calculation.

With Liskov Substitution Principle:

We can separate the concern of bonus calculation into a different interface and only let qualified classes implement it.

```

using System;
namespace LSPDemo
{
    public interface IBonusProvider
    {
        double CalculateBonus();
    }
    public class Employee : IBonusProvider
    {
        public double Salary { get; set; }
        public virtual double CalculateBonus()
        {
            return Salary * 0.1;
        }
    }
    public class Intern
    {
        public double Salary { get; set; }
        // Notice: We don't have a CalculateBonus method here.
    }
}

```

```

//Testing the Liskov Substitution Principle
public class Program
{
    public static void DisplayBonus(IBonusProvider bonusProvider)
    {
        Console.WriteLine($"Bonus: {bonusProvider.CalculateBonus()}");
    }
    public static void Main()
    {
        Employee emp = new Employee { Salary = 5000 };
        DisplayBonus(emp); // Works as expected
        // We no longer have the potential issue of trying to calculate a
an Intern.
        Console.ReadKey();
    }
}
}

```

We ensure that the Liskov Substitution Principle is not breached by only allowing classes that offer bonuses to implement the IBonusProvider interface. This is because an Intern instance will not be required to calculate bonuses.

In the next article, I will discuss the [Interface Segregation Principle in C#](#) with Examples. In this article, I explain **Real-Time Examples of the Liskov Substitution Principle (LSP) in C#**. I hope you enjoy this Liskov Substitution Principle (LSP) Real-Time Examples using the C# article.